

AES-128

Design Specifications

September 2026

TABLE OF CONTENT

1. INTRODUCTION	2
1.1. Introduce about AES-128 algorithm	3
1.2. AES Encryption	3
1.3. AES Key Expansion	5
1.4. AES Decryption	8
2. DESIGN SPECIFICATIONS	10
2.1. AES key features	11
2.2. AES Top Interface	11
2.3. AES High level block design	11
2.4. AES Cipher	12
2.4.1. AES Cipher Core	14
2.4.2. AES Cipher Controller	22
2.4.3. AES Key Expansion block	24
2.5. AES Decipher	28
2.5.1. AES Decipher Core	30
2.5.2. AES Decipher Controller	35
2.5.3. AES Inverse Key Expansion block	37

1. INTRODUCTION

1.1. Introduce about AES-128 algorithm

This standard specifies the Rijndael algorithm ([3] and [4]), a symmetric block cipher that can process data blocks of 128 bits, using cipher keys with lengths of 128, 192, and 256 bits. Rijndael was designed to handle additional block sizes and key lengths, however they are not adopted in this standard.

Throughout the remainder of this standard, the algorithm specified herein will be referred to as “the AES algorithm.” The algorithm may be used with the three different key lengths indicated above, and therefore these different “flavors” may be referred to as “AES-128”, “AES-192”, and “AES-256”.

The Advanced Encryption Standard (AES) is a widely used block cipher algorithm standardized in FIPS-197. It operates on fixed-size data blocks of 128 bits and supports key lengths of 128, 192, or 256 bits.

This specification describes the design of an AES IP core configured for 128-bit key length. The core implements the AES encryption process according to the FIPS-197 standard.

AES performs multiple rounds of transformation on data blocks. The number of rounds varies with the key length. In the 128-bit configuration, the algorithm executes 10 rounds.

	Key length (Nk)	Block size (Nb)	Number of Round (Nr)
AES-128	4	4	10
AES-192	6	4	12
AES-256	6	4	14

1.2. AES Encryption

At the start of the Cipher, the input is copied to the State array using the conventions. After an initial Round Key addition, the State array is transformed by implementing a round function 10, 12, or 14 times (depending on the key length), with the final round differing slightly from the first Nr -1 rounds. The final State is then copied to the output. The Cipher is described in the pseudo code in Figure 1.1. The individual transformations SubBytes(), ShiftRows(), MixColumns(), and AddRoundKey():

```

Cipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
begin
    byte state[4,Nb]

    state = in

    AddRoundKey(state, w[0, Nb-1])                // See Sec. 5.1.4

    for round = 1 step 1 to Nr-1
        SubBytes(state)                            // See Sec. 5.1.1
        ShiftRows(state)                           // See Sec. 5.1.2
        MixColumns(state)                           // See Sec. 5.1.3
        AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])
    end for

    SubBytes(state)
    ShiftRows(state)
    AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])

    out = state
end

```

Figure 1.1. Pseudo Code for the Cipher

AES is also based on two fundamental principles of cryptography: substitution (confusion) and permutation (diffusion), but it implements them differently from DES. Each round consists of four main steps:

- AddRoundKey – XOR the data with the corresponding round key.
- SubBytes – Substitute each byte with another value from the S-box to increase non-linearity.
- ShiftRows – Perform cyclic shifts of the rows in the data matrix to achieve diffusion.
- MixColumns – Mix the data across columns using multiplication in the finite field $GF(2^8)$; this step is omitted in the final round.

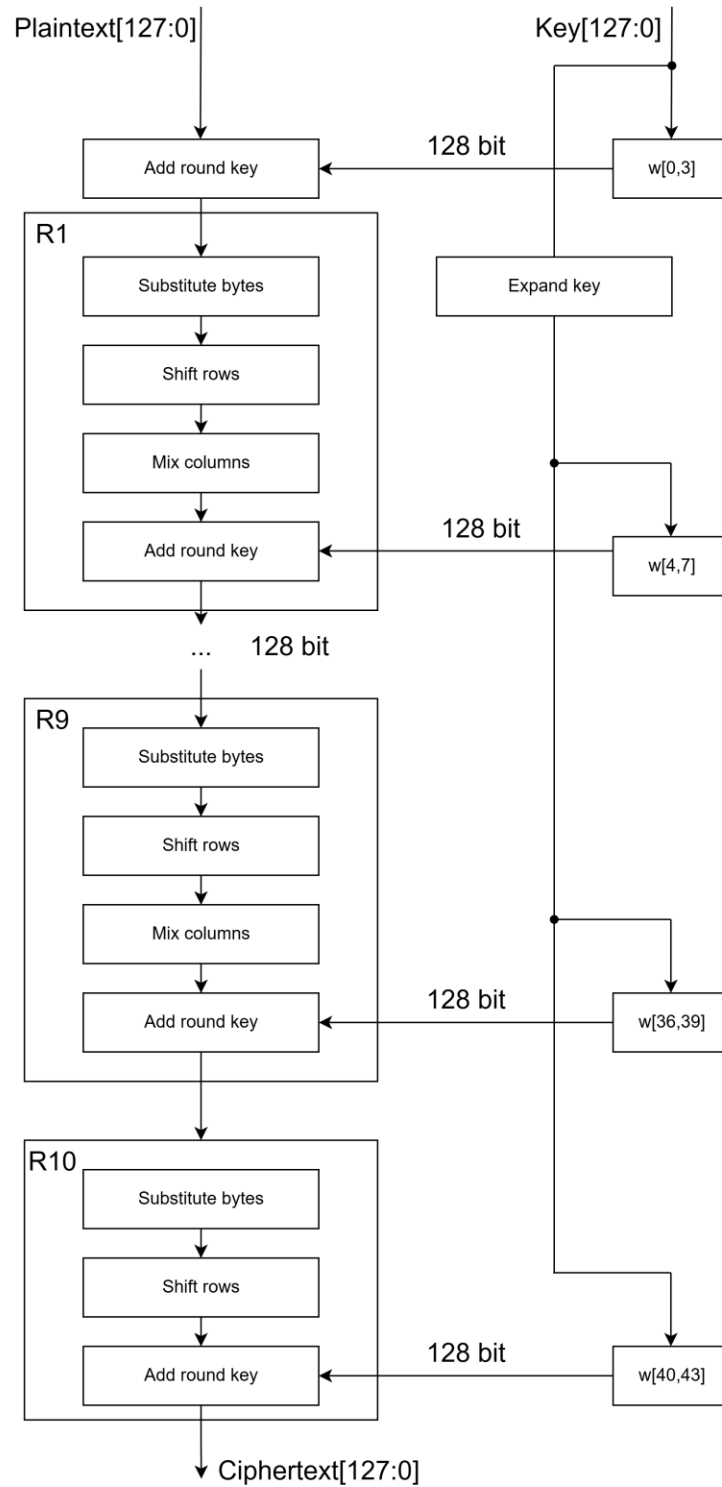


Figure 1.2. The algorithm flow chart for AES Cipher

1.3. AES Key Expansion

The AES algorithm takes the Cipher Key, K , and performs a Key Expansion routine to generate a key schedule. The Key Expansion generates a total of N_b ($N_r + 1$) words: the algorithm requires an initial set of N_b words, and each of the N_r rounds requires N_b words of key data. The resulting key schedule consists of a linear array of 4-byte words, denoted $[w_i]$, with i in the range $0 \leq i < N_b(N_r + 1)$.

The expansion of the input key into the key schedule proceeds according to the pseudo code in Figure 1.3.

```

KeyExpansion(byte key[4*Nk], word w[Nb*(Nr+1)], Nk)
begin
    word temp

    i = 0

    while (i < Nk)
        w[i] = word(key[4*i], key[4*i+1], key[4*i+2], key[4*i+3])
        i = i+1
    end while

    i = Nk

    while (i < Nb * (Nr+1))
        temp = w[i-1]
        if (i mod Nk = 0)
            temp = SubWord(RotWord(temp)) xor Rcon[i/Nk]
        else if (Nk > 6 and i mod Nk = 4)
            temp = SubWord(temp)
        end if
        w[i] = w[i-Nk] xor temp
        i = i + 1
    end while
end

```

Figure 1.3. Pseudo Code for Key Expansion

AES Key Expansion is the process of generating round keys from the initial cipher key. For AES-128, the algorithm produces 11 round keys (one for the initial round and 10 for the encryption rounds). Each step includes:

- Initialization – The original cipher key (128 bits) is divided into 4 words ($N_k = 4$).

- RotWord – Performs a cyclic rotation of a word (e.g., $[a_0, a_1, a_2, a_3] \rightarrow [a_1, a_2, a_3, a_0]$).
- SubWord – Applies the S-box substitution to each byte of the word.
- Rcon – Adds a round constant to introduce non-linearity and uniqueness per round.
- XOR with previous words – Each new word is generated by XORing with a word N_k position earlier.

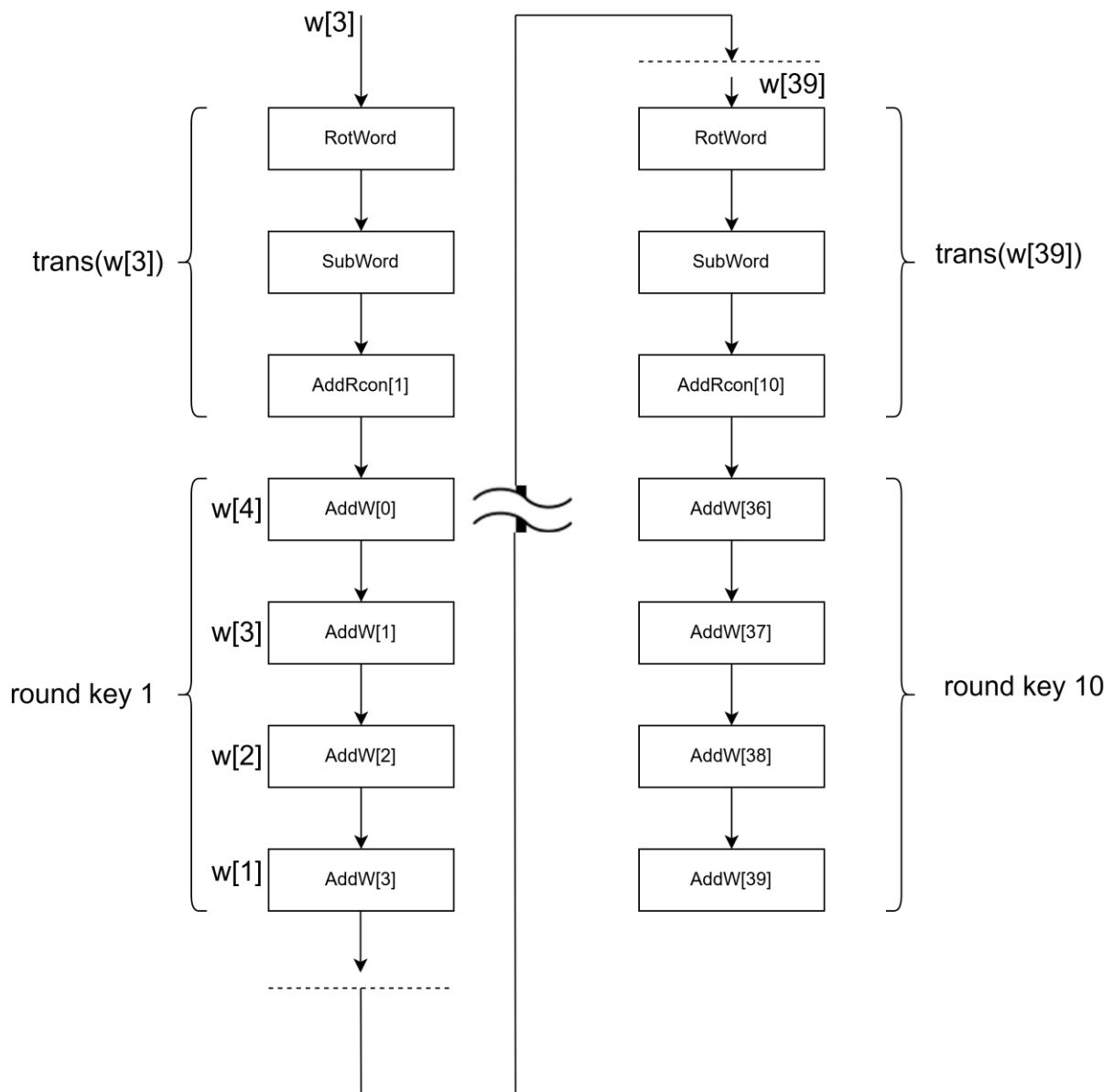


Figure 1.4. The algorithm flow chart for Key expansion

1.4. AES Decryption

The Cipher transformations can be inverted and then implemented in reverse order to produce a straightforward Inverse Cipher for the AES algorithm. The individual transformations used in the Inverse Cipher - InvShiftRows(), InvSubBytes(), InvMixColumns(), and AddRoundKey(). The Inverse Cipher is described in the pseudo code in Figure 1.5.

```

InvCipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
begin
    byte state[4,Nb]

    state = in

    AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1]) // See Sec. 5.1.4

    for round = Nr-1 step -1 downto 1
        InvShiftRows(state) // See Sec. 5.3.1
        InvSubBytes(state) // See Sec. 5.3.2
        AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])
        InvMixColumns(state) // See Sec. 5.3.3
    end for

    InvShiftRows(state)
    InvSubBytes(state)
    AddRoundKey(state, w[0, Nb-1])

    out = state
end

```

Figure 1.5. Pseudo Code for Inverse Cipher

AES Decryption is similar to encryption but performed in reverse order. AES uses the same secret key for both encryption and decryption because it is a symmetric algorithm. Instead of applying the forward transformations, decryption applies the inverse operations to recover the original plaintext from the ciphertext:

- AddRoundKey – XOR the data with the corresponding round key.
- InvMixColumns – Apply the inverse column mixing to undo MixColumns (omitted in the final round).

- InvShiftRows – Perform cyclic shifts in the opposite direction of ShiftRows to restore the previous state.
- InvSubBytes – Substitute each byte using the inverse S-box to reverse the SubBytes transformation.

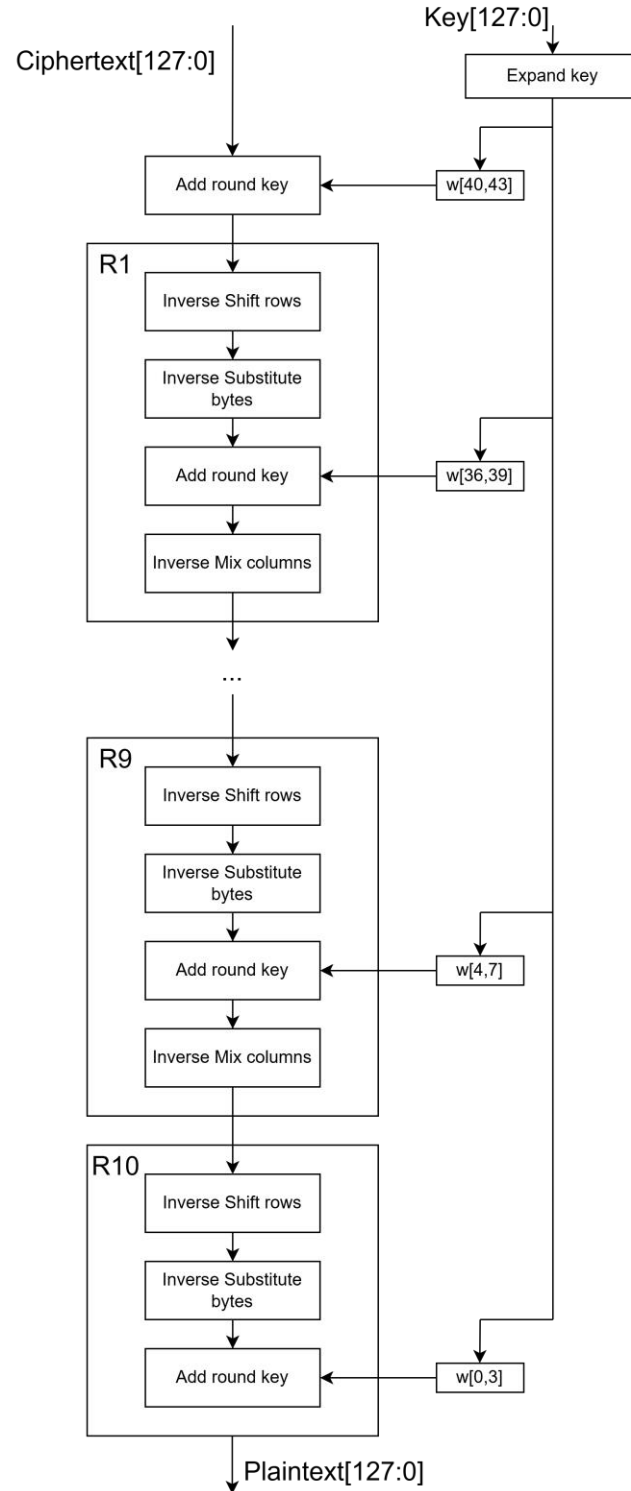


Figure 1.6. The algorithm flow chart for AES Decipher

2. DESIGN SPECIFICATIONS

2.1. AES key features

In this project, the IP AES-128 will implement 2 main functions: Encrypt the plaintext to ciphertext and decrypt the ciphertext to plaintext. At one time, the IP is just set 1 block active (Cipher or Decipher).

2.2. AES Top Interface

The Figure 2.1 below show about the AES Top Interface:

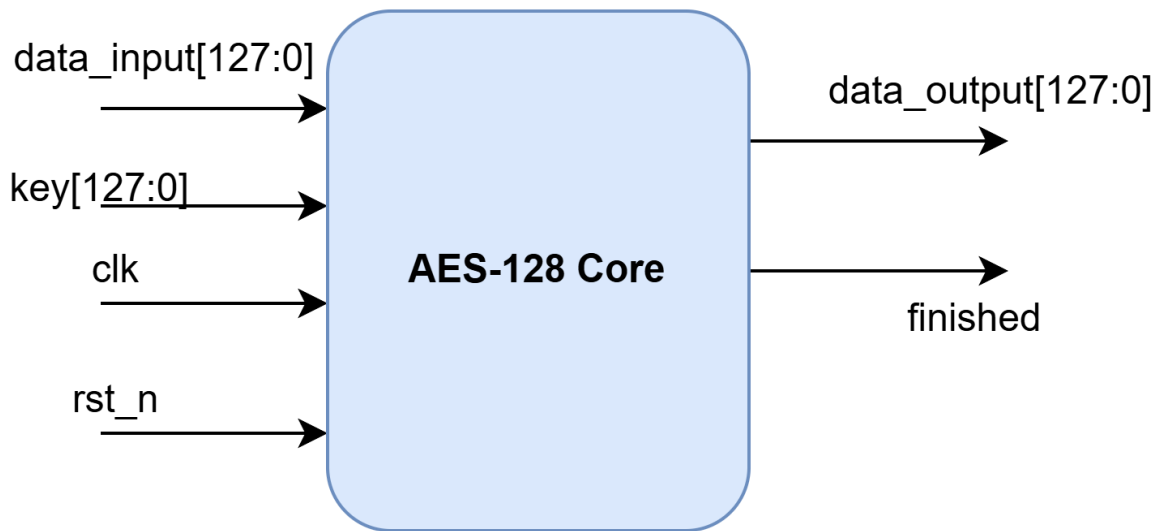


Figure 2.1. Top Interface of IP AES-128

Table 2.1. Table of AES Interface signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
data_input	128	input	Input data need to be processed (encrypt or decrypt)
key	128	input	Input key for encryption or decryption
data_output	128	output	Data output after process (encryption or decryption)
finished	1	output	The finished signal of encryption or decryption

2.3. AES High level block design

Figure 2.2. show about the high-level block of IP AES-128.

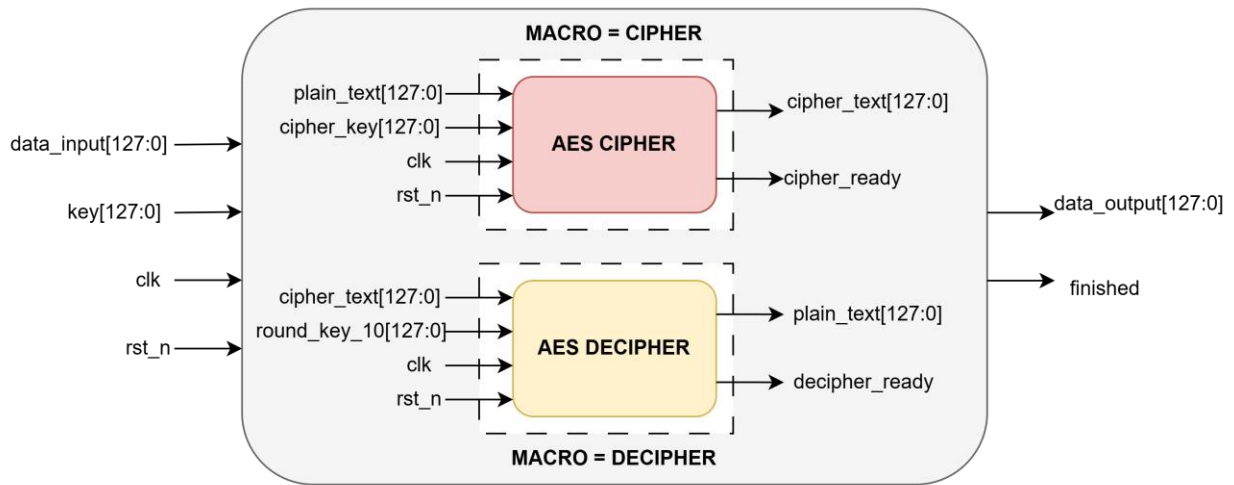


Figure 2.2. High level design of IP AES-128

The AES IP core uses MACRO definitions to configure which functional block is active. This mechanism determines whether the IP operates in encryption (CIPHER) or decryption (DECIPHER) mode.

- If the CIPHER macro is defined, the AES encryption block is enabled.
- If the DECIPHER macro is defined, the AES decryption block is enabled.
- By default, the IP is configured for CIPHER mode.

2.4. AES Cipher

This block performs the encryption process. It accepts a 128-bit plaintext input and produces a 128-bit ciphertext output. In addition, it provides a cipher_ready signal that indicates whether the encryption core is idle or has completed an encryption cycle. The block diagram of AES Cipher is shown as Figure 2.3. and Figure 2.4. below.

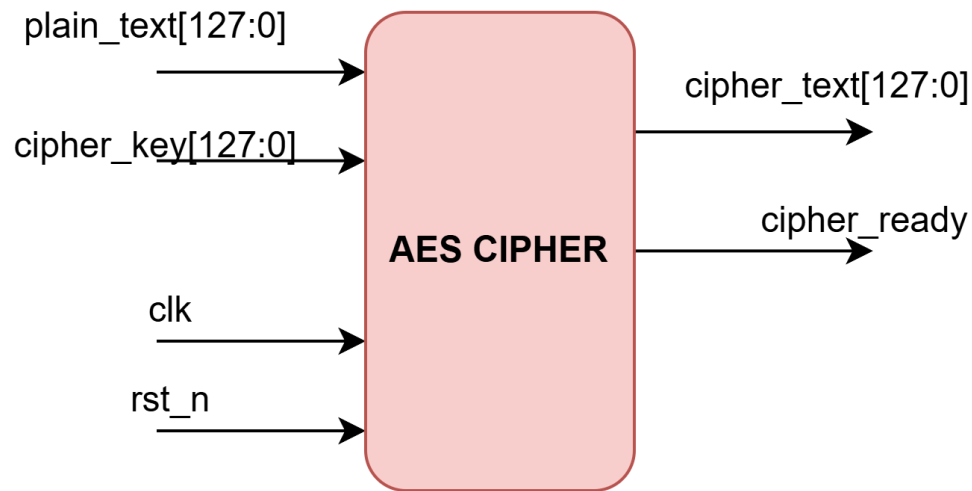


Figure 2.3. AES CIPHER Interface

Table 2.2. Table of AES CIPHER Interface signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
plain_text	128	input	Input data need to be encrypted
cipher_key	128	input	Input key for encryption
cipher_text	128	output	Data output after encryption
cipher_ready	1	output	The finished signal of encryption

For high level block of AES CIPHER, the block diagram will be shown below:

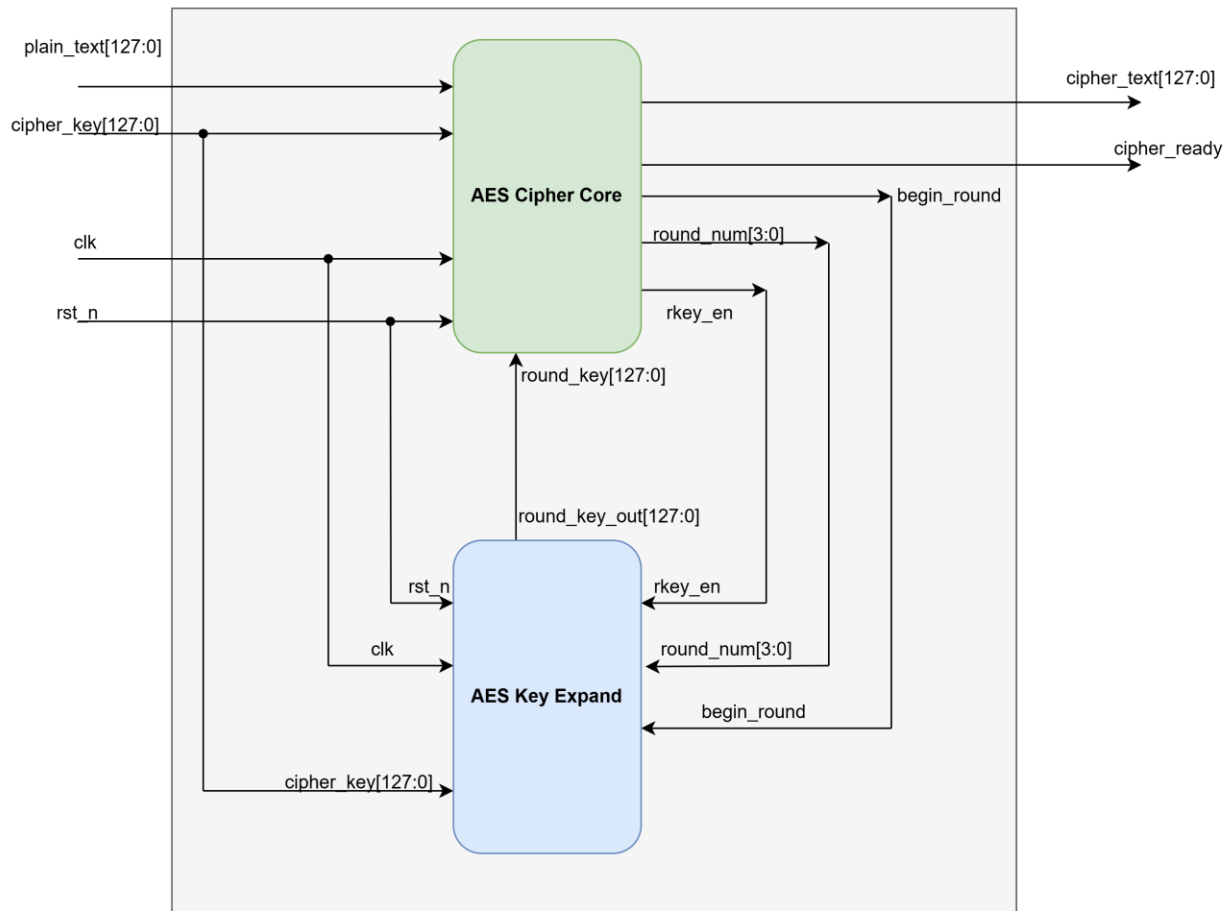


Figure 2.4. The high-level block diagram of AES CIPHER

The AES CIPHER block is composed of two primary components:

- **Cipher Core** – Responsible for the main encryption function, transforming plaintext into ciphertext.
- **Key Expansion Unit** – Generates the round keys (cipher keys) required for each encryption round.

Together, these blocks ensure secure and efficient AES encryption in compliance with the standard.

2.4.1. AES Cipher Core

This block will implement the encryption algorithm of the AES-128 algorithm. The Cipher core block diagram will be shown in Figure 2.5.

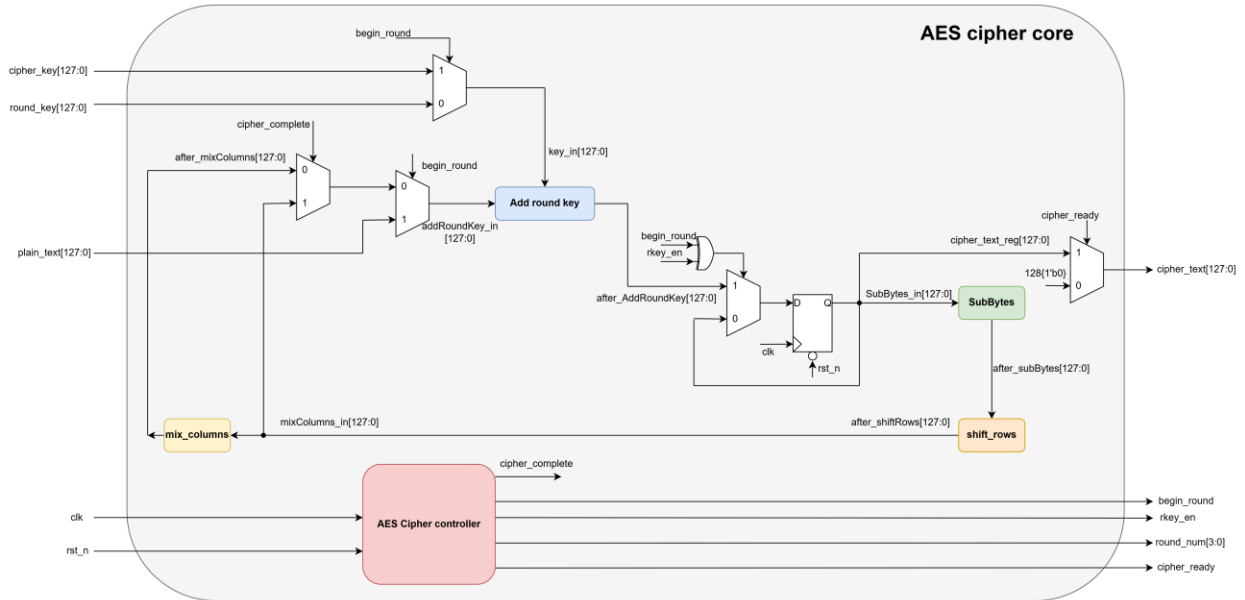


Figure 2.5. The block diagram of AES Cipher core

Table 2.3. Table of the AES Cipher core signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
plain_text	128	input	Input data need to be encrypted
cipher_key	128	input	Input key for encryption
round_key	128	output	The round key from AES Key Expand block (use for the encryption round 1 to 10)
rkey_en	1	output	The enable signal for AES Key Expand to operate
round_num	4	output	Indicates the current encryption round.
begin_round	1	output	The signal indicates the start of the encryption process; <code>begin_round = 1</code> when <code>round_num = 0</code> .
cipher_text	128	output	Data output after encryption
cipher_ready	1	output	The finished signal of encryption

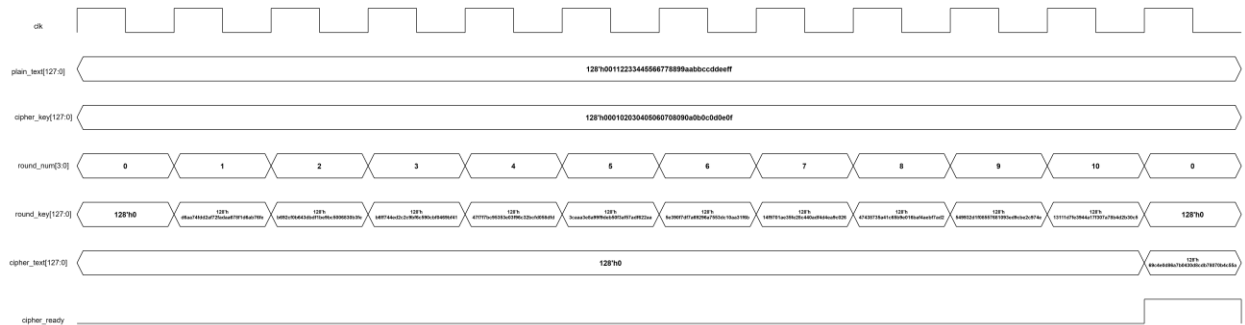


Figure 2.6. The waveform diagram of AES Cipher core

At the waveform diagram of AES Cipher core, the encryption progress must have 10 rounds to implement or encrypt 1 data. When the encryption is finished, the signal cipher_ready will assert to 1 and the data will be captured at this time.

a. AddRoundKey

Function: Combine the current state with the round key using the XOR operation, ensuring that the encryption key is integrated into the data.

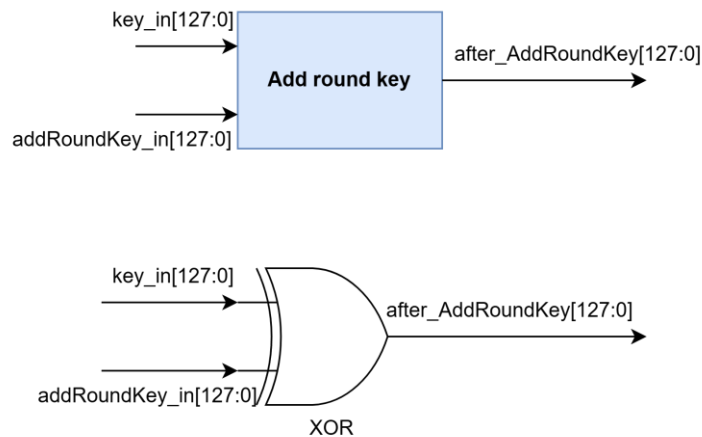


Figure 2.7. The AddRoundKey block

b. SubBytes

Function: Apply a non-linear substitution using the S-Box, ensuring non-linearity to resist linear and differential attacks.

From SBox table of Figure 2.8. We have the block design of SubBytes in Figure 2.9.

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Figure 2.8. SBox Table

As for the S-Box table, the 128-bit block is divided into 16 components of 8 bits each. For every 8-bit value, the upper 4 bits are used to determine the row index, and the lower 4 bits are used to determine the column index. This allows the corresponding value to be referenced directly from the S-Box.

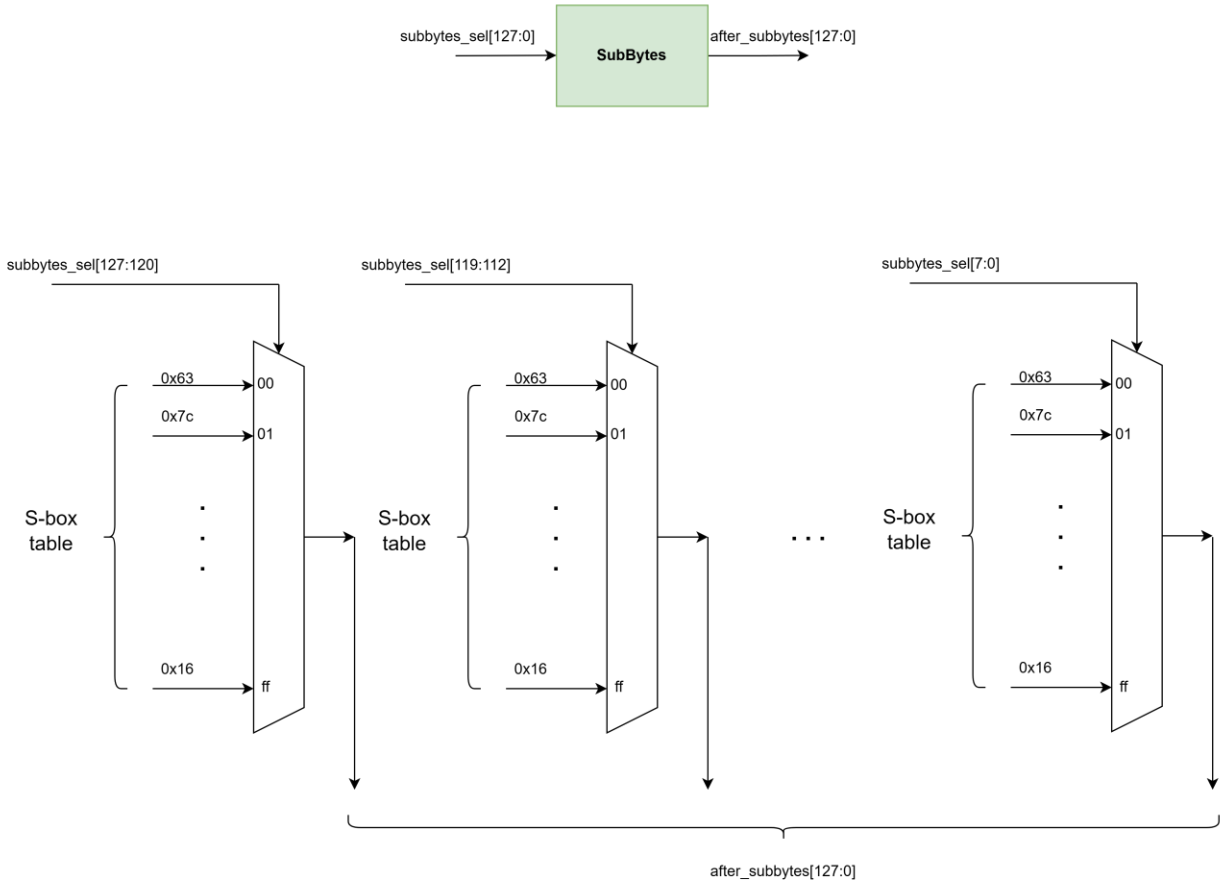


Figure 2.9. SubBytes block design

c. ShiftRows

Function: Perform cyclic shifts on each row of the state, providing diffusion during the encryption process.

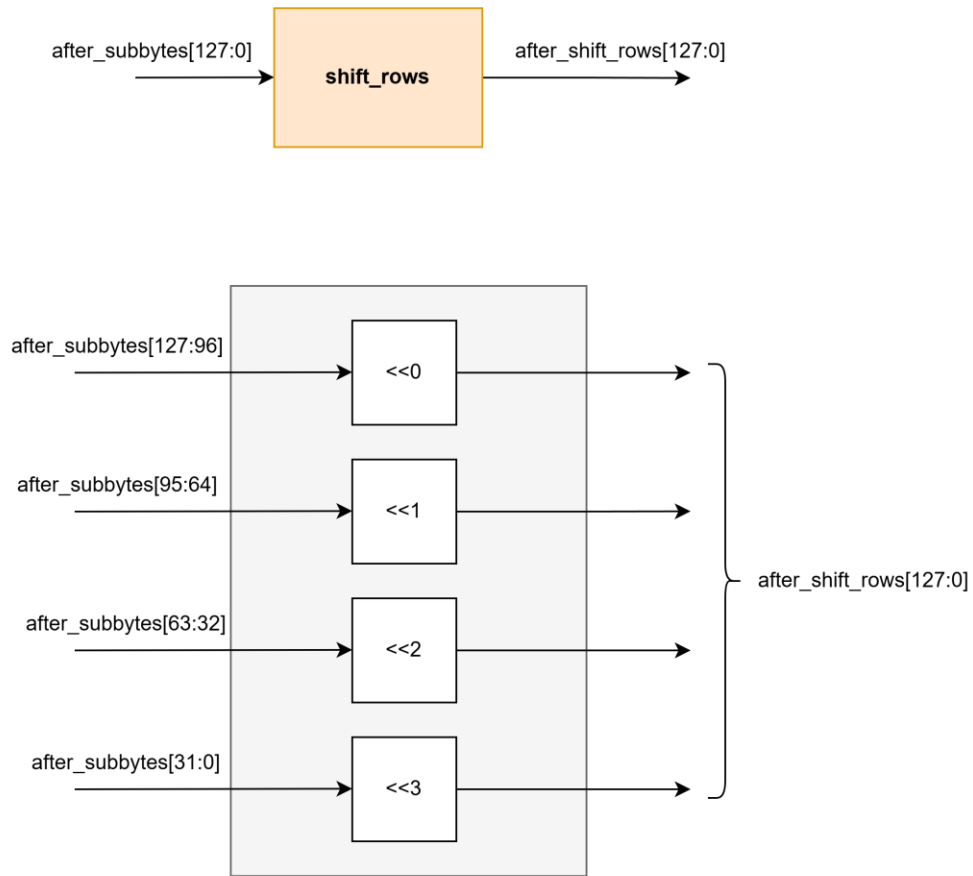


Figure 2.10. ShiftRows block

d. MixColumns

Function: Combine the bytes within a column using linear transformations, enhancing diffusion by mixing the column data.

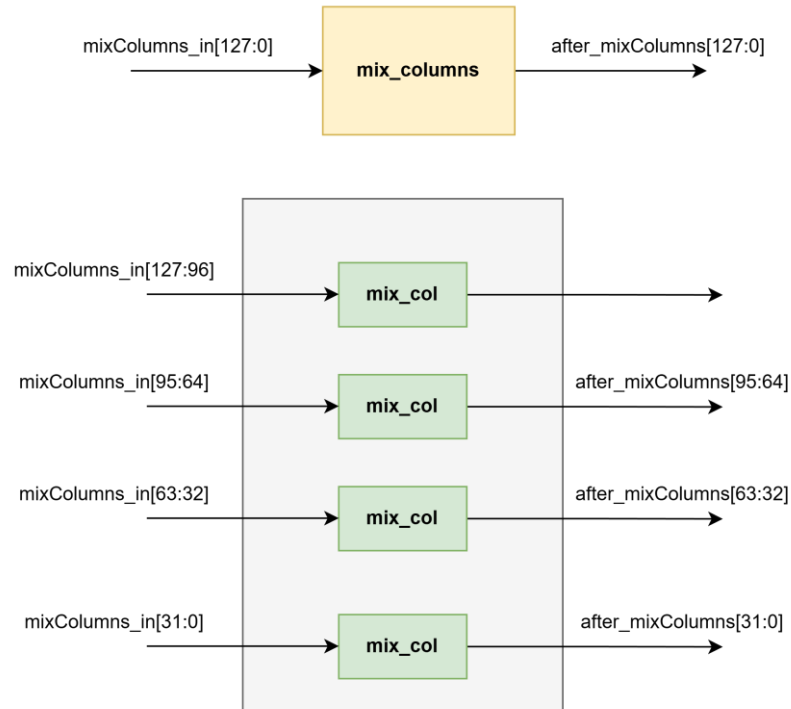


Figure 2.11. MixColumn block

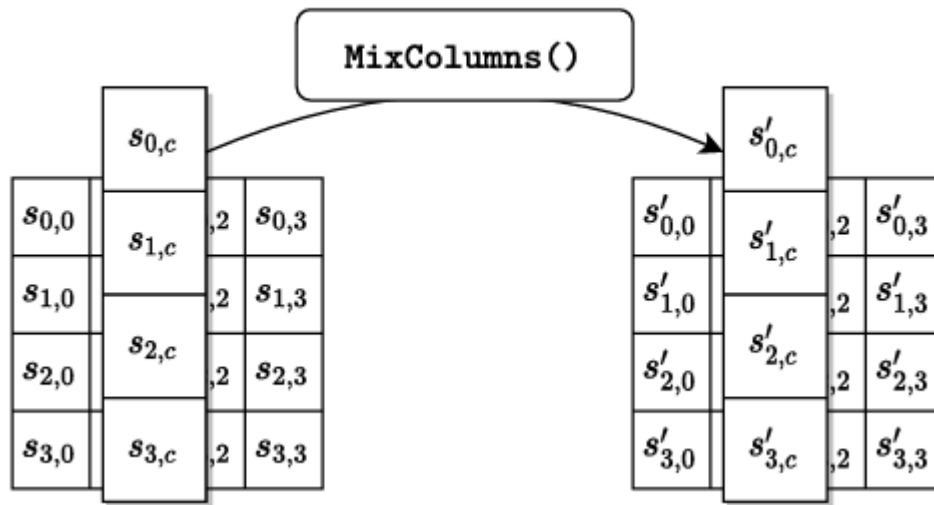


Figure 2.12. Illustration of MIXCOLUMNS()

MIXCOLUMNS() is a transformation of the state that multiplies each of the four columns of the state by a single fixed matrix, with its entries taken from the following word:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad \text{for } 0 \leq c < 4,$$

so that the individual output bytes are defined as follows:

$$\begin{aligned} s'_{0,c} &= (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c} \\ s'_{1,c} &= s_{0,c} \oplus (\{02\} \bullet s_{1,c}) \oplus (\{03\} \bullet s_{2,c}) \oplus s_{3,c} \\ s'_{2,c} &= s_{0,c} \oplus s_{1,c} \oplus (\{02\} \bullet s_{2,c}) \oplus (\{03\} \bullet s_{3,c}) \\ s'_{3,c} &= (\{03\} \bullet s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \bullet s_{3,c}). \end{aligned}$$

The principle of generating multiplication values for bytes is as follows:

- Multiplication by H01 keeps the original value unchanged.
- Multiplication by H02 shifts the byte left by one bit and XORs with H1B if the most significant bit (MSB) is 1.
- Multiplication by H03 is the XOR of the result of multiplication by H01 and the result of multiplication by H02.



Figure 2.13. mix_col block

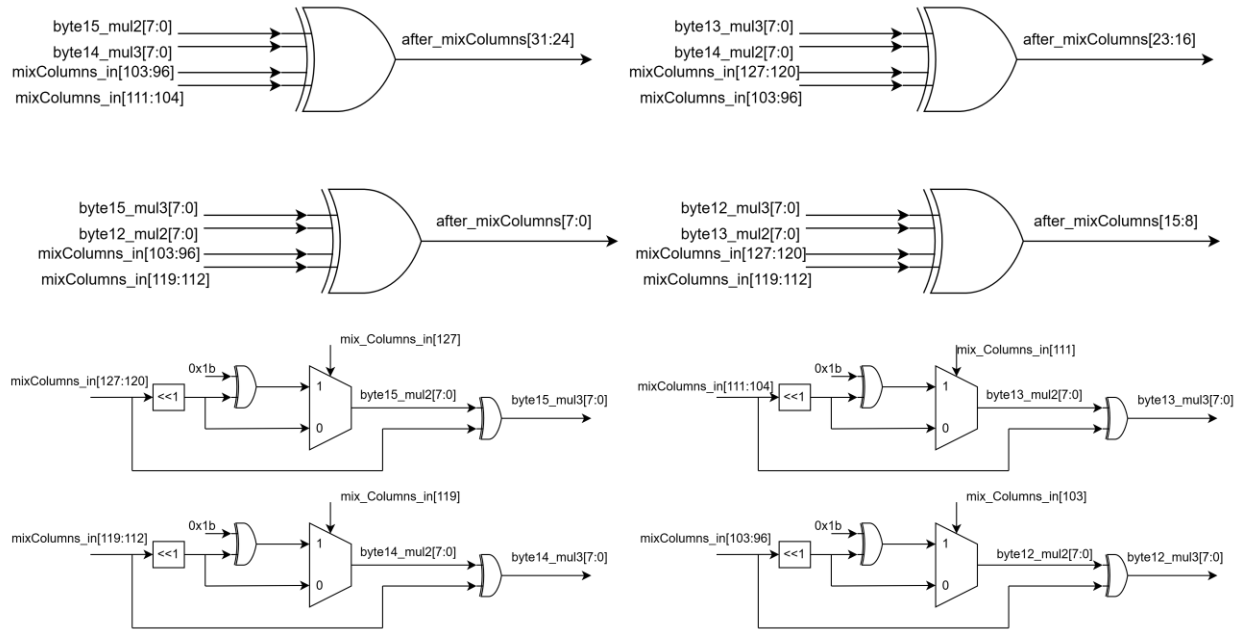


Figure 2.14. Low level block design of `mix_col` block

2.4.2. AES Cipher Controller

Function: This block generates the control signals for Cipher core and Key Expand block. This block will control the encryption process in IP Core.

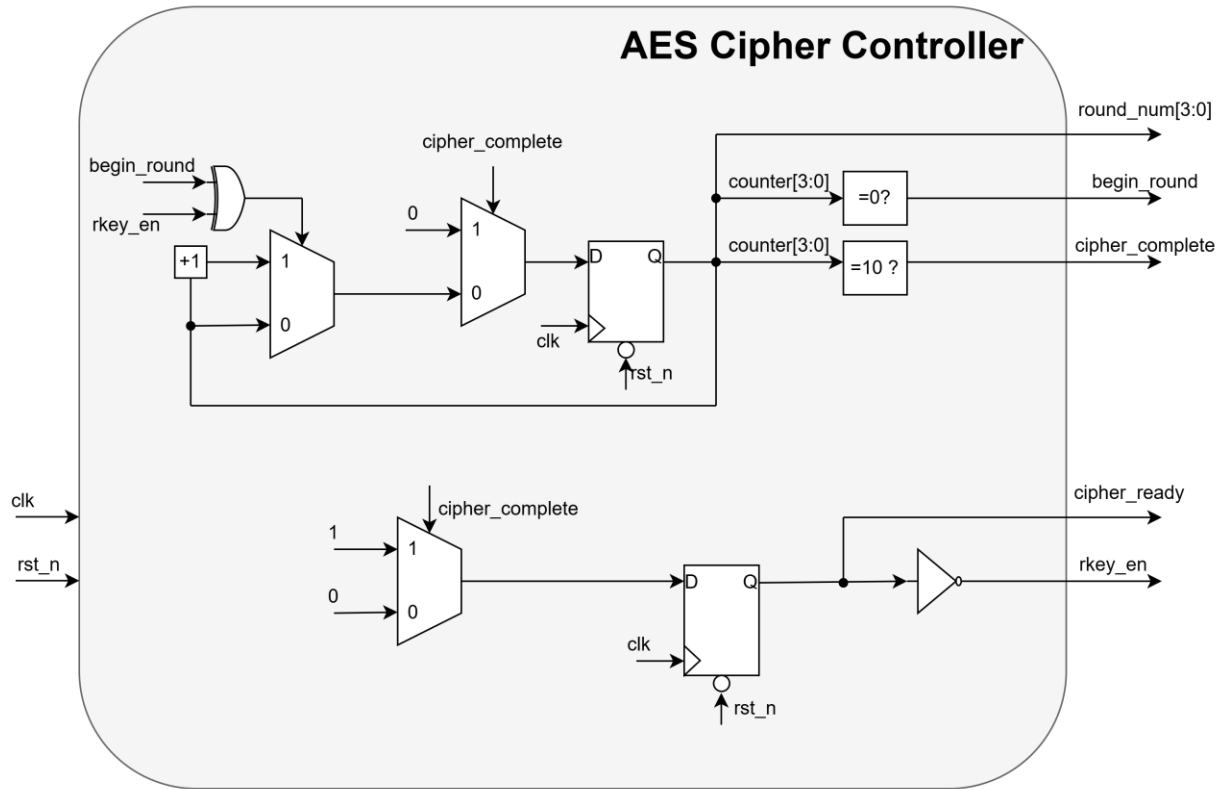


Figure 2.15. The AES Cipher Controller

Table 2.4. Table of AES Cipher Controller signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
round_num	4	output	Indicates the current encryption round.
begin_round	1	output	The signal indicates the start of the encryption process; begin_round = 1 when round_num = 0.
cipher_complete	1	output	The signal indicates the final encryption round, meaning the encryption is nearly complete. cipher_complete = 1 when round_num = 10
cipher_ready	1	output	The finished signal of encryption

rkey_en	1	output	The enable signal for AES Key Expand to operate
---------	---	--------	---

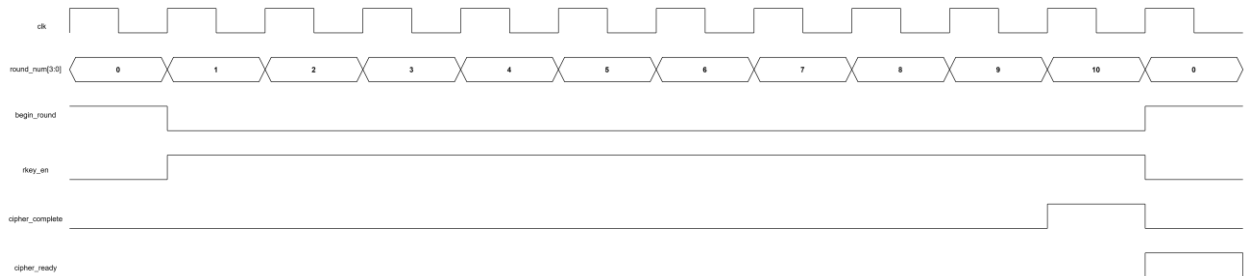


Figure 2.16. The waveform diagram of AES Cipher Controller

2.4.3. AES Key Expansion block

This block performs the function of generating round keys for the encryption of rounds (from round 1 to round 10).

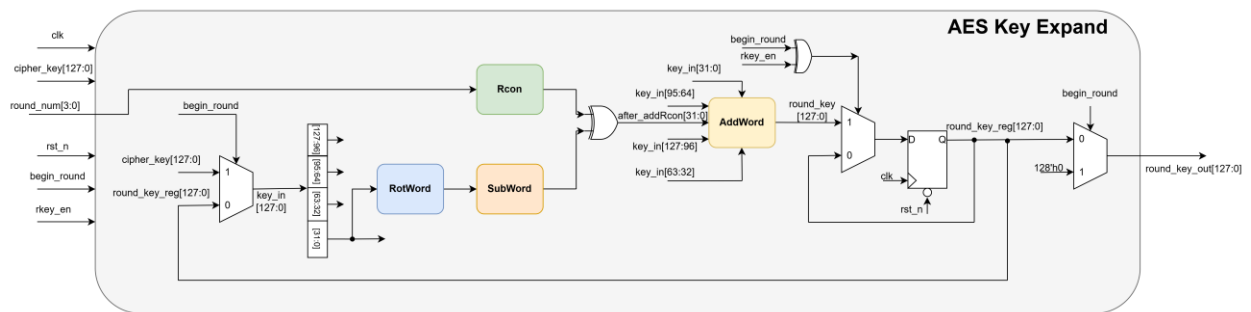


Figure 2.17. AES Key Expand block

Table 2.5. Table of AES Key Expand signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
round_num	4	input	Indicates the current encryption round.
rkey_en	1	input	The enable signal for AES Key Expand to operate
cipher_key	128	input	Input key for encryption
begin_round	1	input	The signal indicates the start of the encryption process; begin_round = 1 when round_num = 0.
round_key_out	128	output	The round key from AES Key Expand block (use for the encryption round 1 to 10)

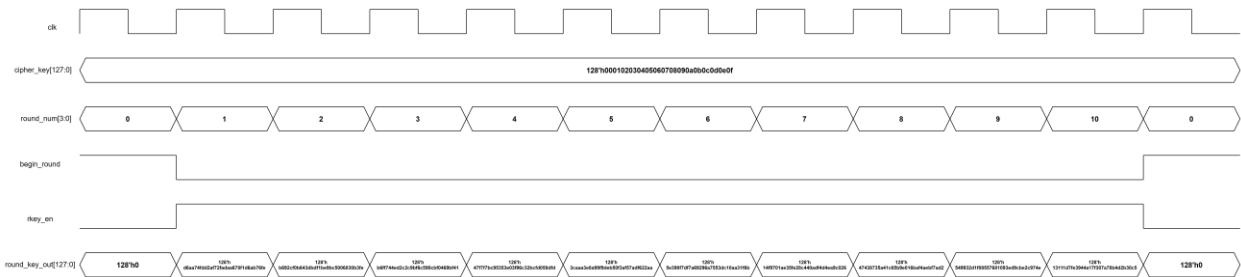


Figure 2.18. The waveform of AES Key Expand block

a. RotWord

Function: The RotWord function performs a left rotation of a word $w[j]$ 1 byte.

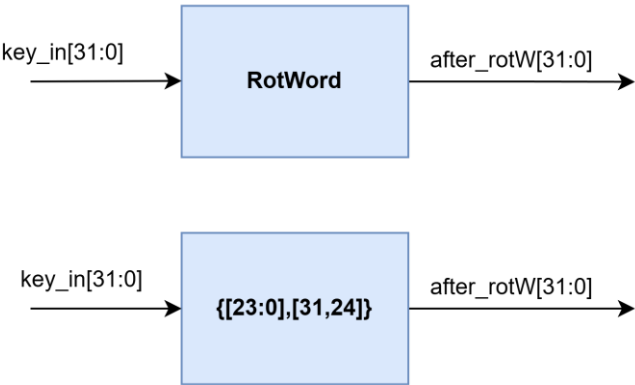


Figure 2.18. RotWord block

b. SubWord

Function: used in the Key Expansion process, it takes a four-byte input word and applies the S-Box substitution to each byte, producing a four-byte output word.

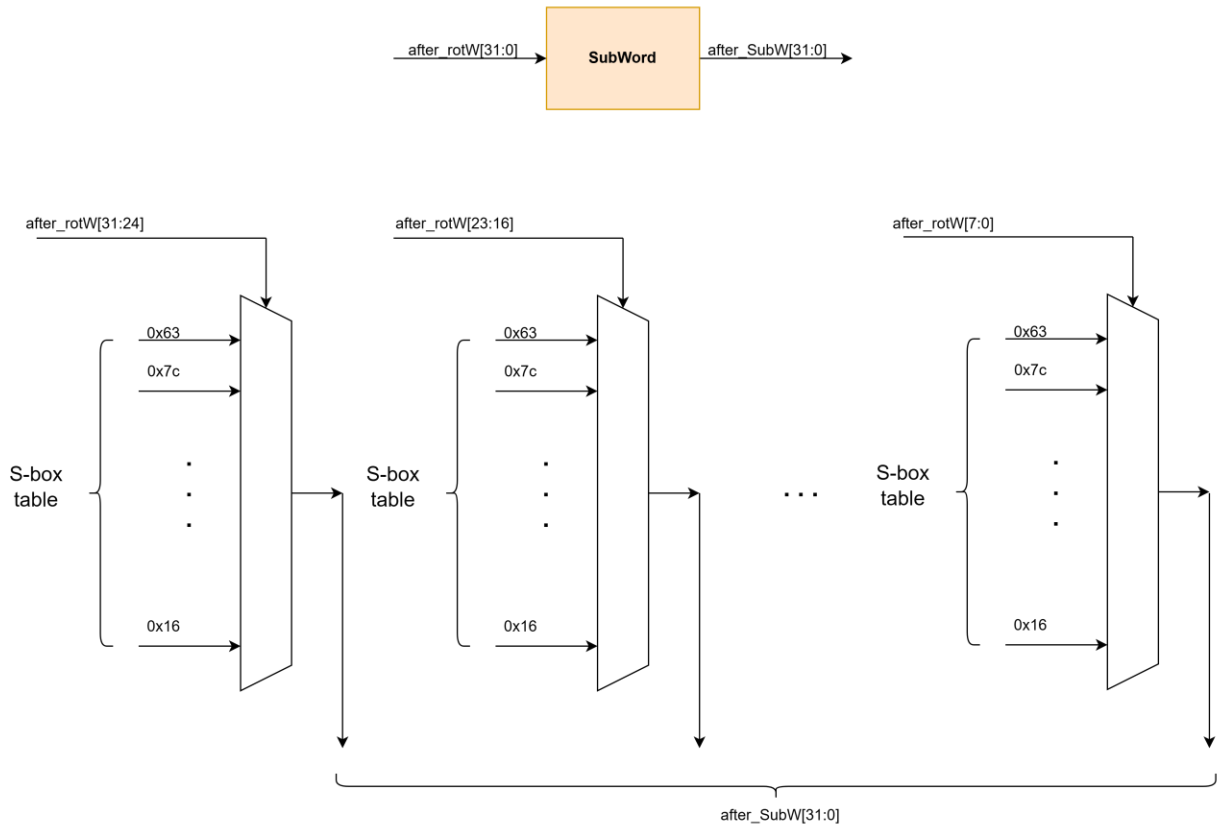


Figure 2.19. SubWord block

c. Rcon

Function: the round constant array ensures that each round key in AES encryption is unique and cannot be derived through simple patterns.

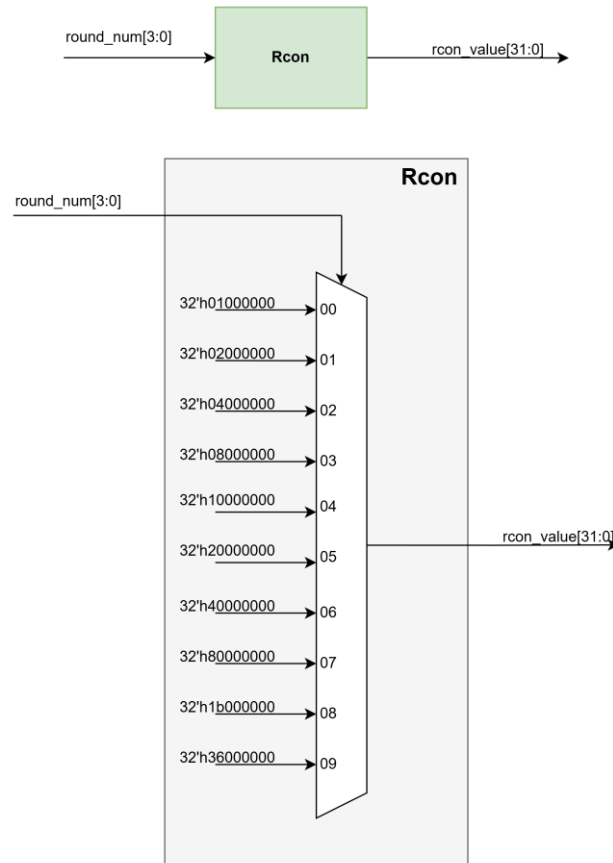


Figure 2.20. Rcon block

d. AddWord

Function: perform bitwise XOR operations between 32-bit words.

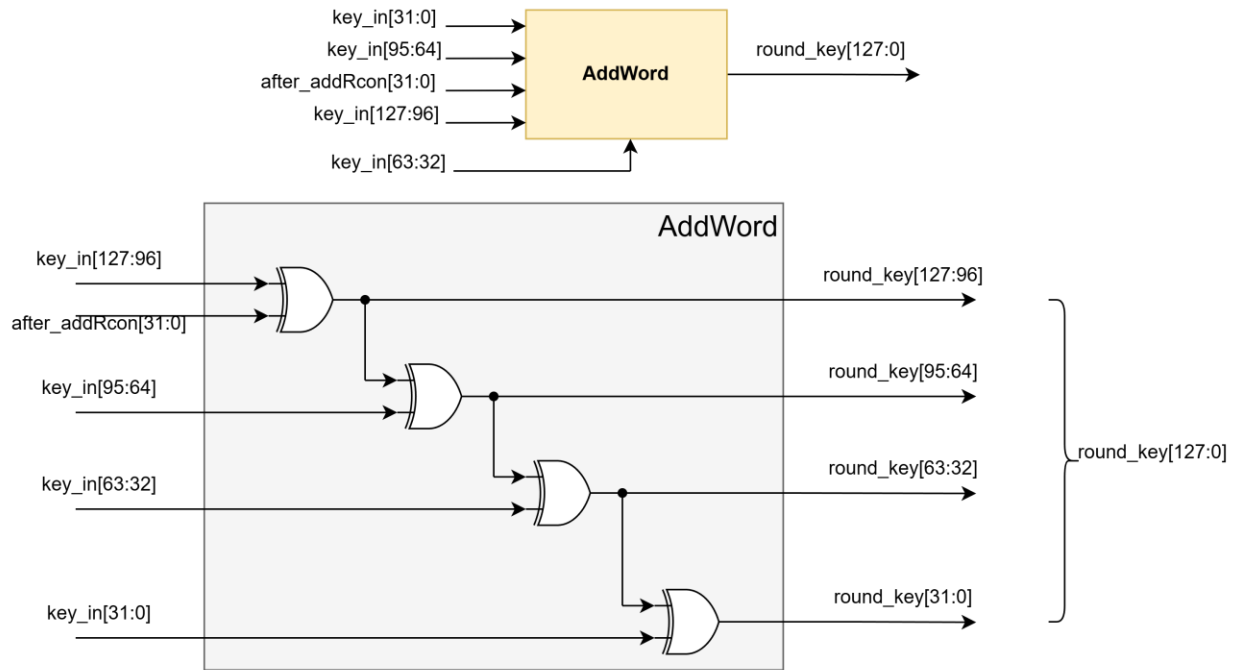


Figure 2.21. AddWord block

2.5. AES Decipher

This block performs data decryption. It takes a 128-bit input of encrypted data and outputs the 128-bit decrypted data, along with a decipher_ready signal indicating whether the encryption core is idle or has completed a decryption cycle.

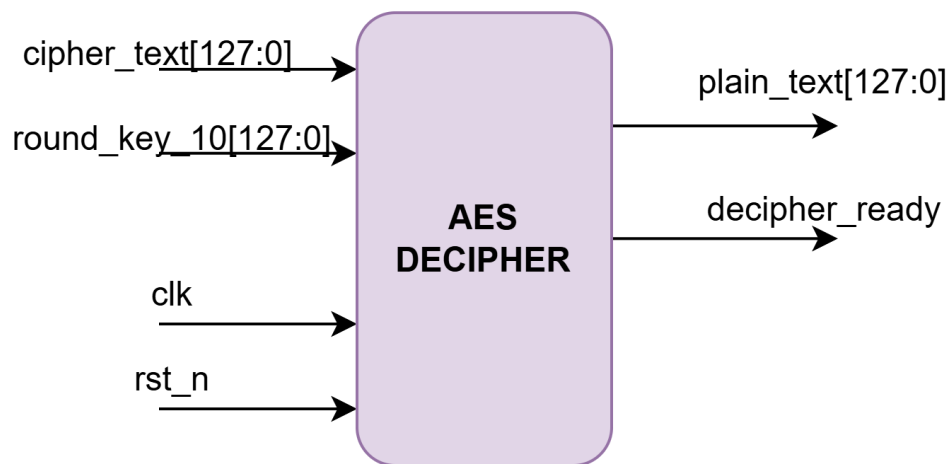


Figure 2.22. AES DECIPHER Interface

Table 2.6. Table of AES DECIPHER Interface signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
cipher_text	128	input	Input data need to be decrypted
round_key_10	128	input	Input key for decryption (Round key 10th of encryption)
plain_text	128	output	Data output after decryption
decipher_ready	1	output	The finished signal of decryption

For high level block of AES DECIPHER, the block diagram will be shown below:

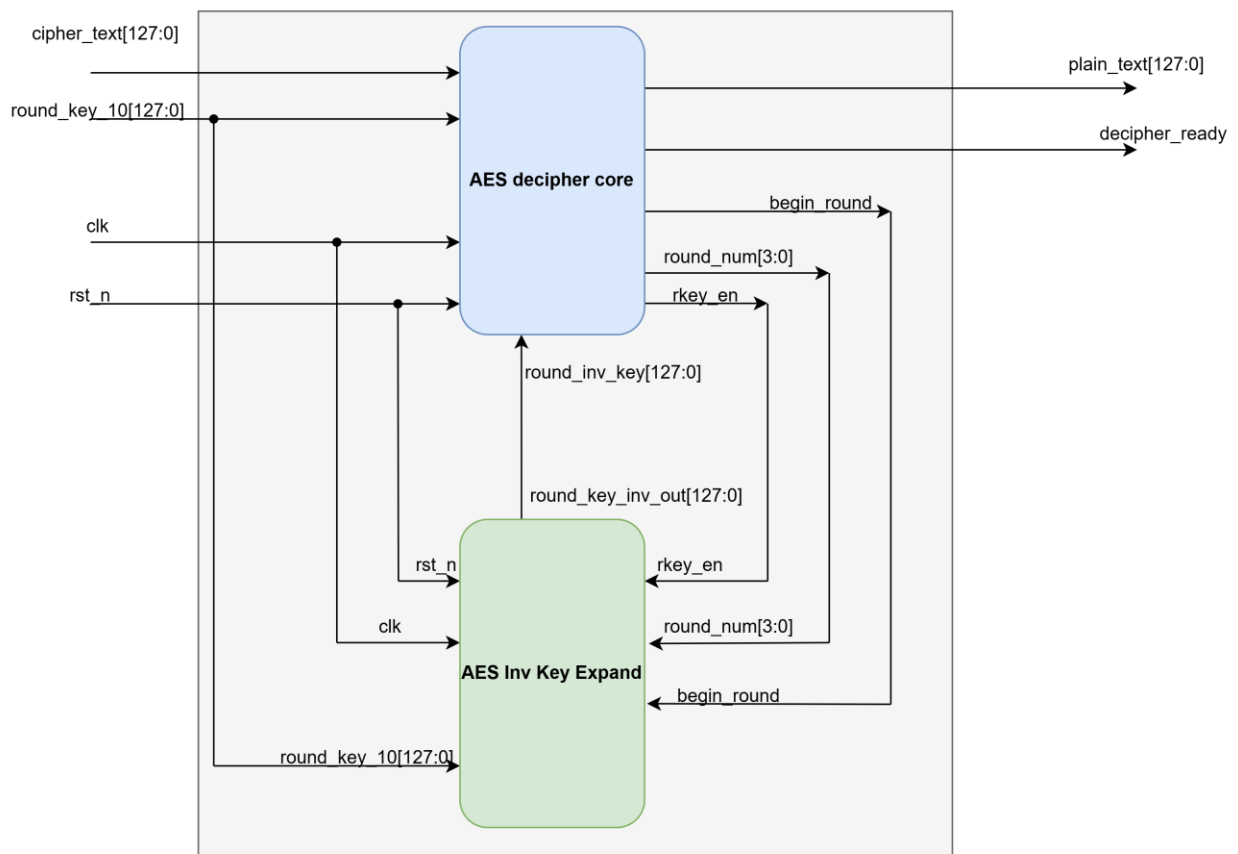


Figure 2.23. AES DECIPHER high-level block design

The AES DECIPHER block is composed of two primary components:

- Cipher Core – Responsible for the main decryption function, transforming ciphertext into plaintext.

- Inverse Key Expansion Unit – Generates the round keys (decipher keys) required for each decryption round.

Together, these blocks ensure secure and efficient AES decryption in compliance with the standard.

2.5.1. AES Decipher Core

This block executes the functional operations of the AES-128 decryption algorithm.

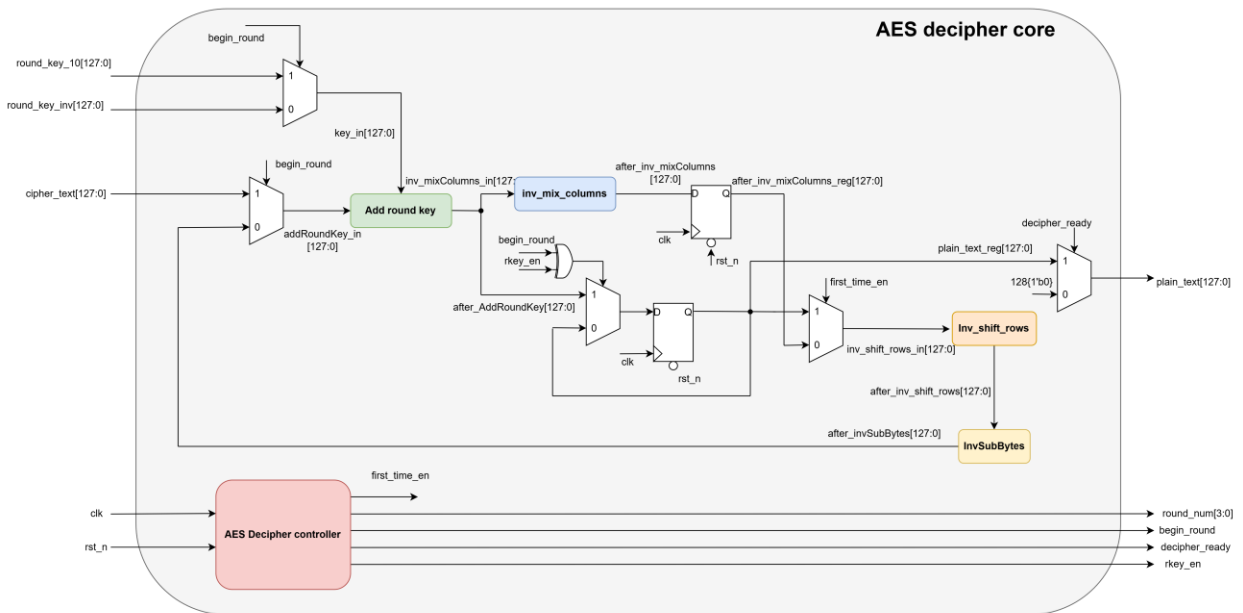


Figure 2.24. AES decipher core

Table 2.7. Table of AES Decipher core signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
cipher_text	128	input	Input data need to be decrypted
round_key_10	128	input	Input key for decryption (Round key 10th of encryption)
round_key_inv	128		The round key values from the inverse round key generation block are used for the decryption rounds from round 1 to round 10.
begin_round	1	input	
plain_text	128	output	Data output after decryption

decipher_ready	1	output	The finished signal of decryption
----------------	---	--------	-----------------------------------

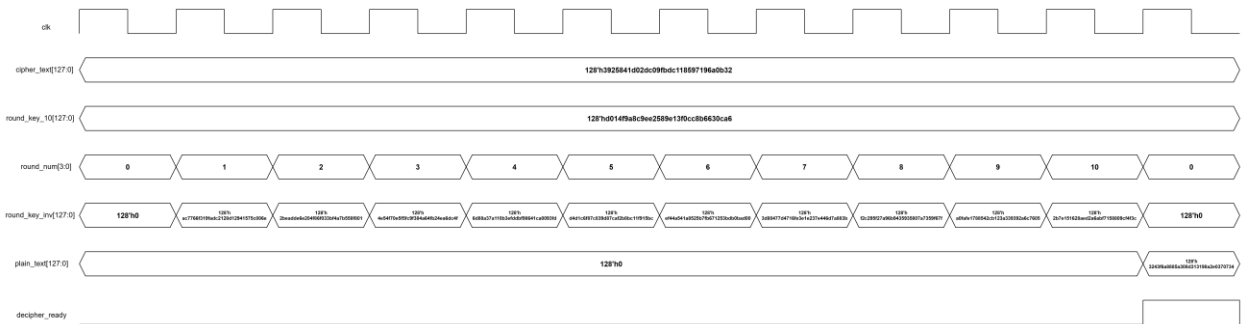


Figure 2.25. The waveform of AES decipher

At the waveform diagram of AES Deipher core, the decryption progress must have 10 rounds to implement or decrypt 1 data. When the decryption is finished, the signal decipher_ready will assert to 1 and the data will be captured at this time.

a. AddRoundKey

Function: combine the current state with the round key using the XOR operation, ensuring the decryption key is applied to the data.

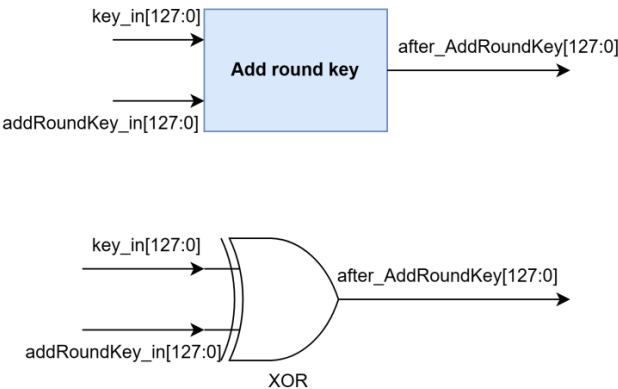


Figure 2.26. AddRoundKey

b. InvShiftRows

Function: the inverse of the ShiftRows block, using the inverse S-Box to restore the original byte values of the state matrix.

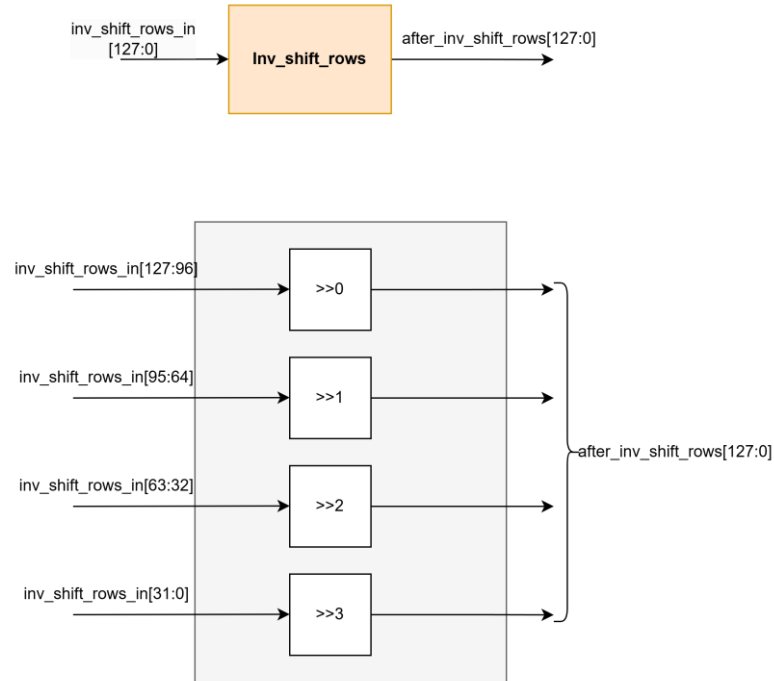


Figure 2.27. *InvShiftRows* block

c. **InvSubBytes**

Function: the inverse of the SubBytes block, using the inverse S-Box to restore the original byte values before the SubBytes transformation.

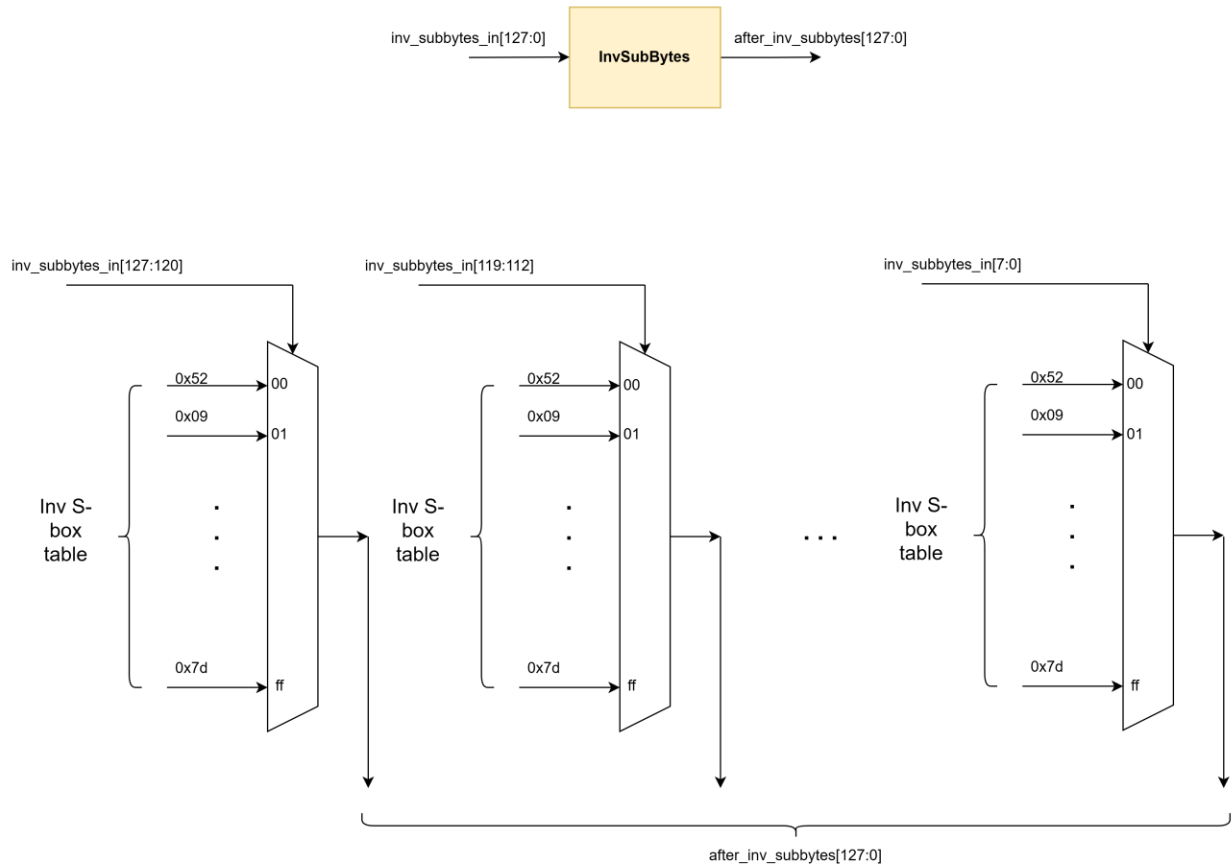


Figure 2.28. *InvSubBytes block*

d. InvMixColumns

Function: the inverse of the MixColumns block, restoring the state to its original form before the MixColumns transformation.

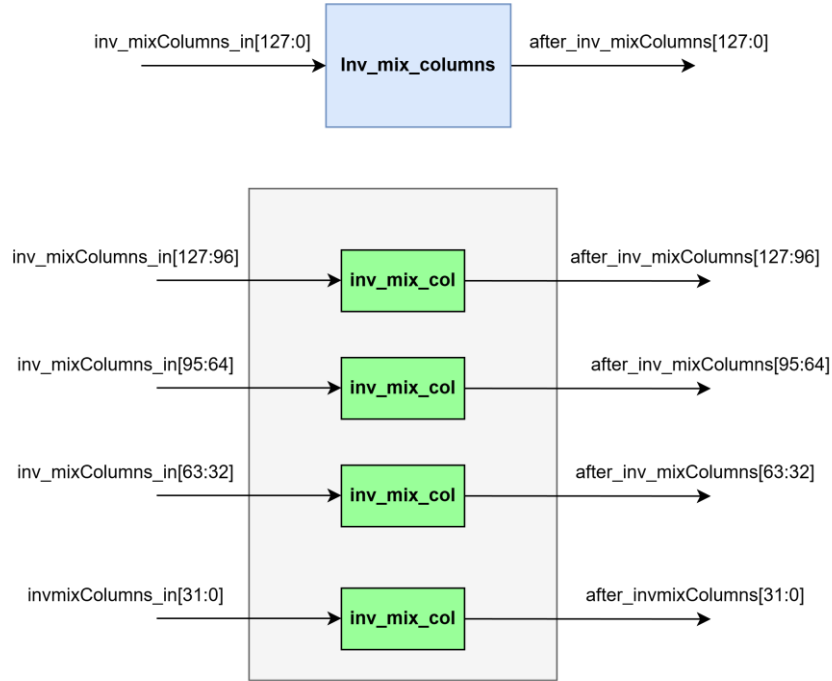


Figure 2.29. InvMixColumns block

INVMIXCOLUMNS() is the inverse of MIXCOLUMNS(). In particular, INVMIXCOLUMNS() multiplies each of the four columns of the state by a single fixed matrix, with its entries taken from the following word:

$$[a0, a1, a2, a3] = [\{0e\}, \{09\}, \{0d\}, \{0b\}].$$

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad \text{for } 0 \leq c < 4.$$

As a result of this matrix multiplication, the four bytes in a column are replaced by the following:

$$\begin{aligned} s'_{0,c} &= (\{0e\} \bullet s_{0,c}) \oplus (\{0b\} \bullet s_{1,c}) \oplus (\{0d\} \bullet s_{2,c}) \oplus (\{09\} \bullet s_{3,c}) \\ s'_{1,c} &= (\{09\} \bullet s_{0,c}) \oplus (\{0e\} \bullet s_{1,c}) \oplus (\{0b\} \bullet s_{2,c}) \oplus (\{0d\} \bullet s_{3,c}) \\ s'_{2,c} &= (\{0d\} \bullet s_{0,c}) \oplus (\{09\} \bullet s_{1,c}) \oplus (\{0e\} \bullet s_{2,c}) \oplus (\{0b\} \bullet s_{3,c}) \\ s'_{3,c} &= (\{0b\} \bullet s_{0,c}) \oplus (\{0d\} \bullet s_{1,c}) \oplus (\{09\} \bullet s_{2,c}) \oplus (\{0e\} \bullet s_{3,c}). \end{aligned}$$

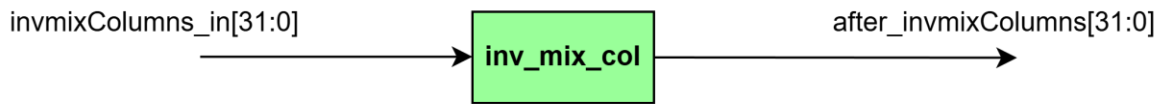


Figure 2.30. *inv_mix_col* blocks

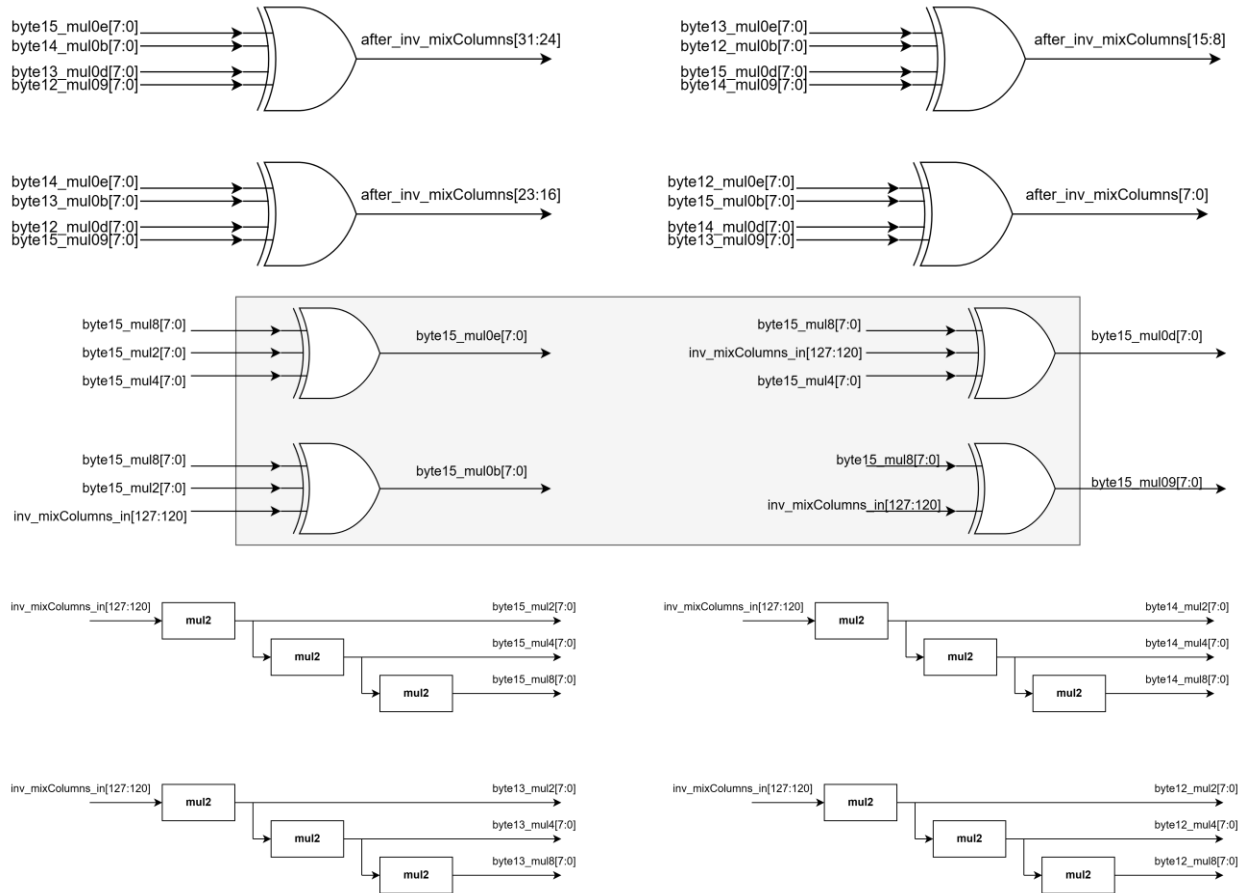


Figure 2.31. *inv_mix_col* low-level block design

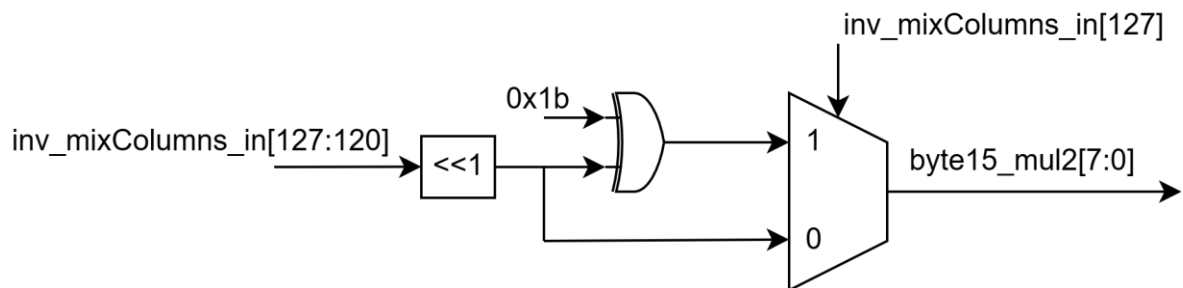


Figure 2.32. *mul2* low level block design

2.5.2. AES Decipher Controller

Function: This block generates the control signals for the Decipher core and Inverse Key Expand block. This block will control the decryption process in IP Core.

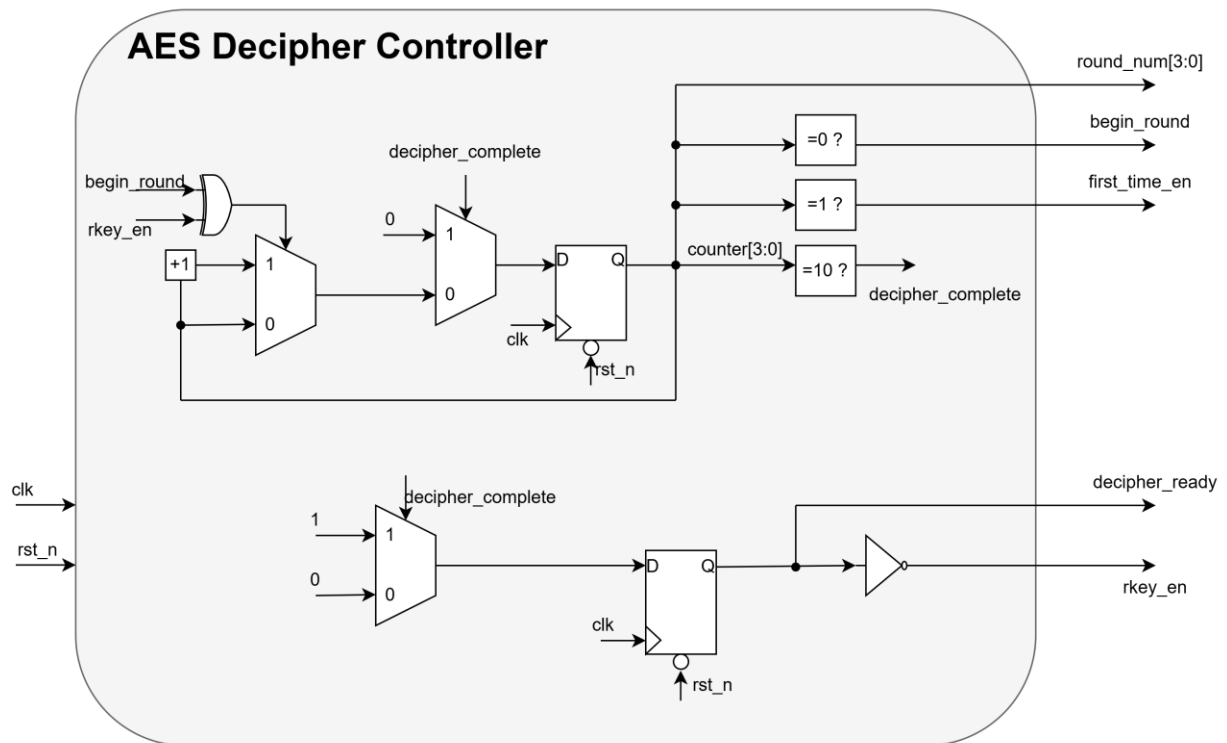


Figure 2.33. AES Decipher Controller

Table 2.8. Table of AES Decipher Controller signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
round_num	4	output	Indicates the current decryption round.
begin_round	1	output	The signal indicates the start of the decryption process; begin_round = 1 when round_num = 0.
decipher_complete	1	output	The signal indicates the final decryption round, meaning the decryption is nearly complete. decipher_complete = 1 when round_num = 10
decipher_ready	1	output	The finished signal of decryption

rkey_en	1	output	The enable signal for AES Inverse Key Expand to operate
---------	---	--------	---

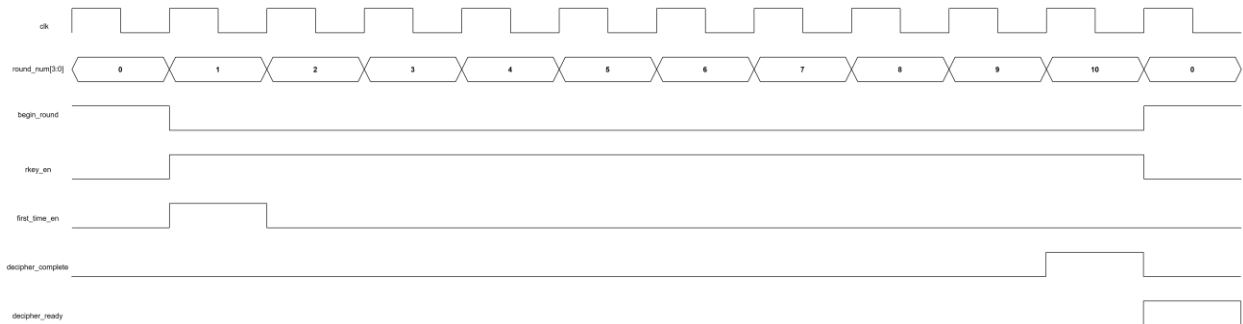


Figure 2.4. The waveform of AES Decipher Controller

2.5.3. AES Inverse Key Expansion block

This block performs the function of generating round keys for the decryption of rounds (from round 1 to round 10).

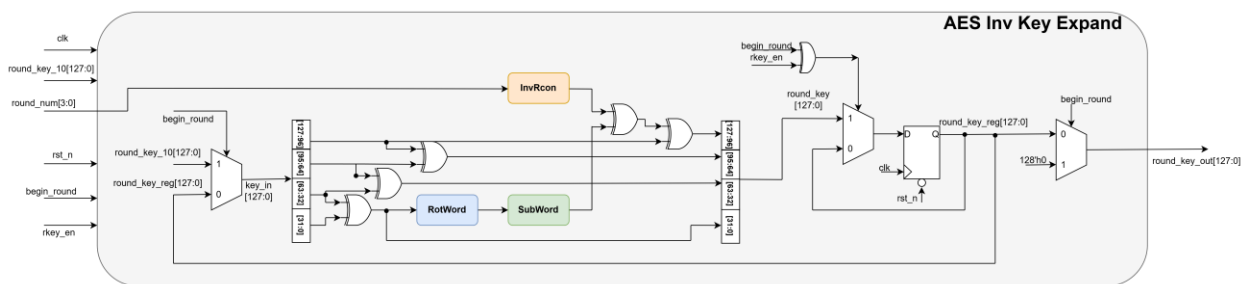


Figure 2.34. AES Inv Key Expand

Table 2.9. Table of AES Inverse Key Expand signals

Port	Size	Direction	Description
clk	1	input	clock
rst_n	1	input	Asynchronous reset, active low
round_num	4	input	Indicates the current encryption round.
rkey_en	1	input	The enable signal for AES Key Expand to operate
round_key_10	128	input	Input key for decryption (Round key 10th of encryption)
begin_round	1	input	The signal indicates the start of the encryption process; begin_round = 1 when round_num = 0.

round_key_out	128	output	The round key from AES Key Expand block (use for the encryption round 1 to 10)
---------------	-----	--------	--

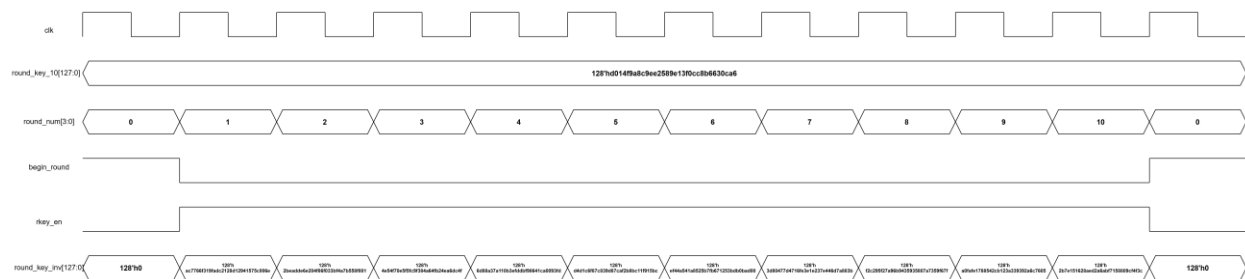


Figure 2.35. The waveform of AES Inverse Key Expand

a. RotWord

Function: The RotWord function performs a left rotation of a word $w[j]$ 1 byte.

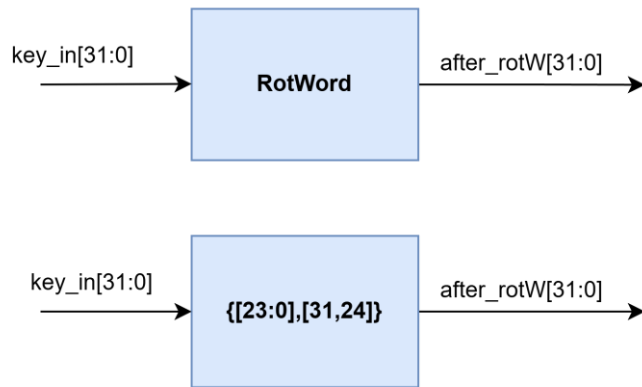


Figure 2.36. RotWord block

b. SubWord

Function: used in the Key Expansion process, it takes a four-byte input word and applies the Inverse S-Box substitution to each byte, producing a four-byte output word.

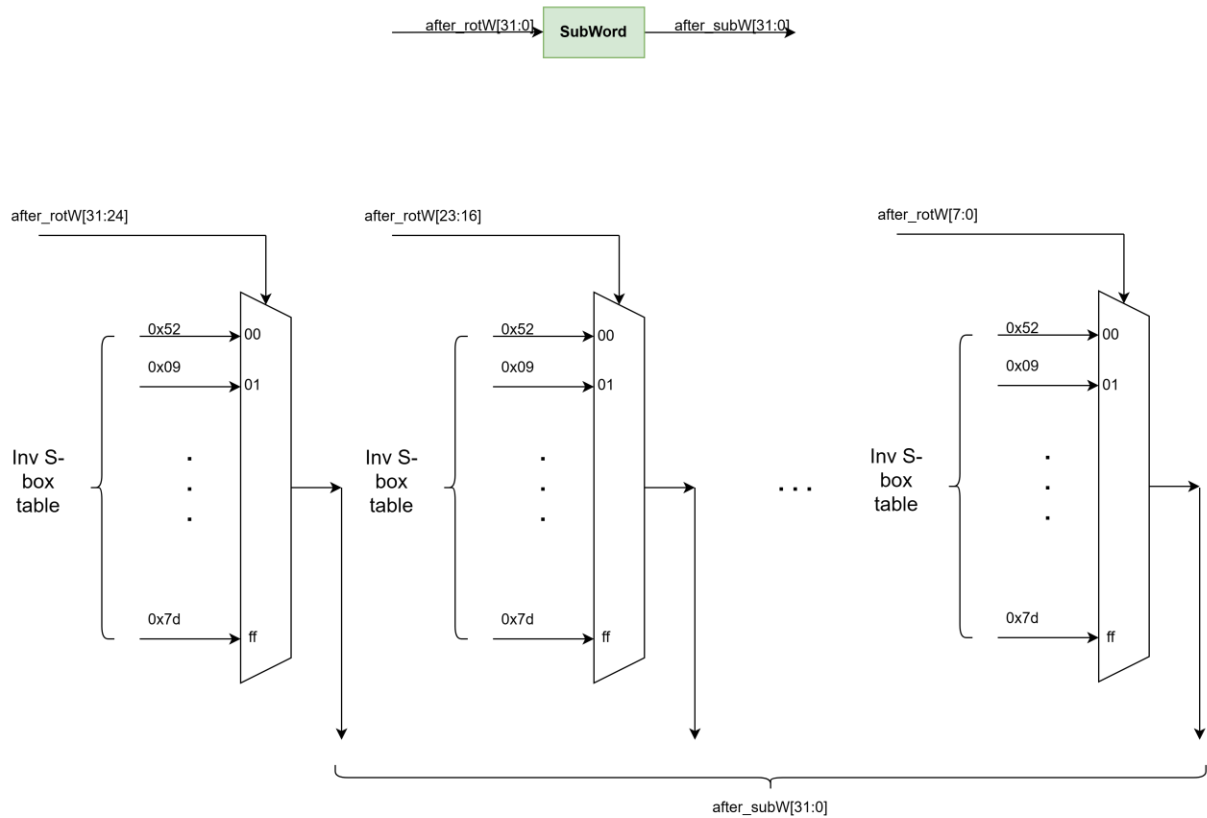


Figure 2.37. SubWord blocks

c. InvRcon

Function: the round constant array ensures that each round key in AES decryption is unique and cannot be derived through simple patterns.

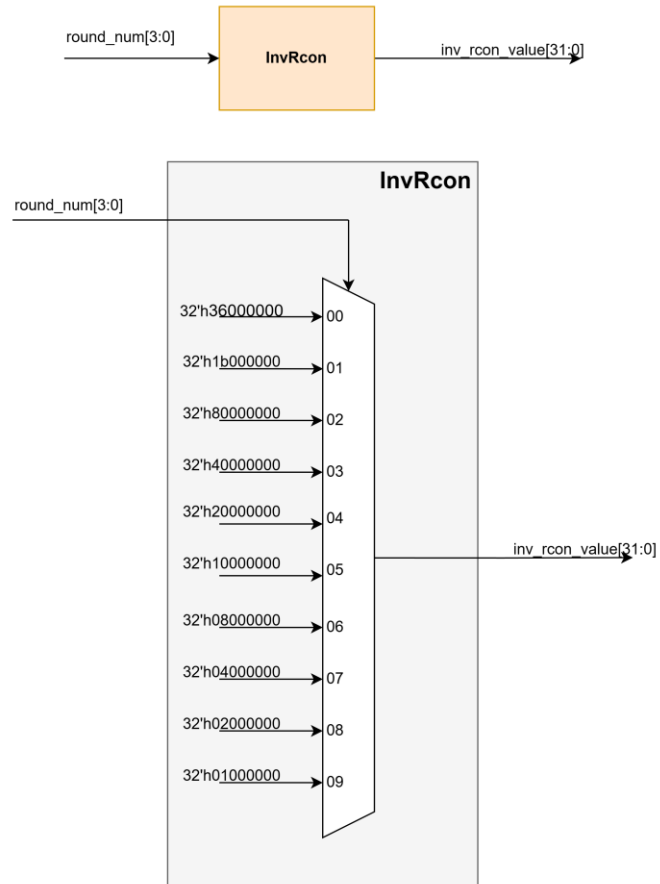


Figure 2.38. InvRcon block